

Insurance Claims Analyzer

Architecture Overview

AI-powered damage assessment prototype

Date: 2026-04-28

Stack: Next.js 16 · TypeScript · Tailwind 4 · Anthropic Claude Sonnet 4.6

This document describes the architecture of a single-image claims-assessment prototype. A single Anthropic Claude API call drives the entire AI pipeline — vision-based vehicle identification, damage classification with severity, and a confidence-bracketed repair-cost estimate — with structured outputs, adaptive thinking, and live streaming back to the user.

1. Design Explanation

Focused answer to the three required sub-bullets of the take-home brief: why these tools, how the AI logic works, and what would change with more time.

Why these tools

Next.js (App Router) + TypeScript + Tailwind in a single repo, because the spec demands a working full-stack demo in 3–4 hours. Next.js gives one codebase, one dev command, one test surface — no client/server split to maintain. Tailwind keeps styling decisions out of the critical path.

Anthropic Claude (Sonnet 4.6) as the only AI service, because the entire pipeline — vehicle identification, damage classification, cost estimation — fits inside a single API call when you combine vision, structured outputs, and adaptive thinking. No model chaining, no glue code, one provider in the architecture diagram. This is the most impactful design decision in the prototype: every other choice flows from it.

Anthropic Files API for image storage, to avoid introducing S3 / Vercel Blob and the associated CORS / presigned-URL plumbing for a 4-hour build. The `file_id` is a durable handle that sets up the multi-step workflows in Phase 2 (reanalysis, before/after comparison, fraud signals) without an architectural pivot. **sharp** handles image preprocessing — resize to ≤ 2576 px and EXIF strip — at the server edge before the image hits Anthropic's storage (real PII removal: GPS, timestamps, device info). **zod** validates Claude's structured output at runtime, so the frontend never has to defensively parse model responses.

How the AI logic works

A single `client.beta.messages.stream()` call drives everything. The image (referenced by `file_id` from the Files API upload) goes into a multimodal message alongside an instruction to analyze. The model is steered by a system prompt that frames Claude as an experienced auto damage adjuster with a five-step playbook (identify vehicle → describe damage → estimate cost → score image quality → flag for review). The system prompt also inlines a typical-cost reference table by damage type and vehicle tier, so cost estimates are anchored in industry-typical ranges.

`output_config.format` enforces a JSON schema on the response — the shape is guaranteed at the API level, no parsing flakiness possible. Adaptive thinking with effort: low lets Claude reason as deeply as it needs but stay tuned for fast first-pass assessment. The endpoint is a Server-Sent Events stream, so the user sees each phase appear live (preprocessing → upload → vehicle ID → output generation). Total wall-clock is ~5–10 seconds, but it feels instant.

After Claude returns, the server validates the JSON with Zod, then runs four deterministic rules to compute typed Trigger objects (low confidence, severe damage, poor image quality, high-value claim). These are the human-review escalation signals — the AI's self-assessment is a fallback; the typed triggers are the source of truth. Each trigger carries a code for routing, the actual value, and the threshold that tripped — machine-readable and auditable.

Potential improvements

The full roadmap is in SOW.md as five phases. Highlights:

Phase / Theme	Description
Persistence + audit log (Phase 1)	Postgres for claim records, S3 + KMS for images with 7-year retention, full read/write audit trail with the Claude model version captured for every analysis.
Production cost integration (Phase 1.5)	Replace the in-prompt reference table with Mitchell / CCC ONE / Audatex APIs for parts-level pricing — the actual stack every major US carrier uses.

Phase / Theme	Description
Multi-image and comparison flows (Phase 2)	Multi-angle upload, side-by-side and AI-powered before/after comparison for repair verification. Introduces an interactive adjuster co-pilot built on Anthropic's Managed Agents API — persistent stateful sessions per claim where adjusters can reanalyze, generate PDF/DOCX claim documents, and iteratively refine the assessment.
Fraud signals (Phase 3)	Image-hash deduplication, visual-similarity scoring against historical claims, AI pairwise comparison on top suspects.
PII redaction pipeline (Phase 1+)	Automatic license-plate and face redaction before storage, configurable retention with archival, full SOC 2 / GDPR-equivalent audit logging.

Smaller wins not in the prototype but easy adds: VIN sticker detection with NHTSA cross-reference (fraud signal), self-consistency sampling on low-confidence cases, cross-model verification on >\$10k claims, and a real claims-system integration (Guidewire / Duck Creek webhook).

2. System Diagram

Three layers — browser, server, external services — connected by a multipart upload (downward) and a Server-Sent Events stream (upward). The right-side panel shows a real sample of the SSE event stream the server emits to the browser as Claude works.

Insurance Claims Analyzer – Architecture

Single Claude API call · Vision + Structured Outputs + Streaming

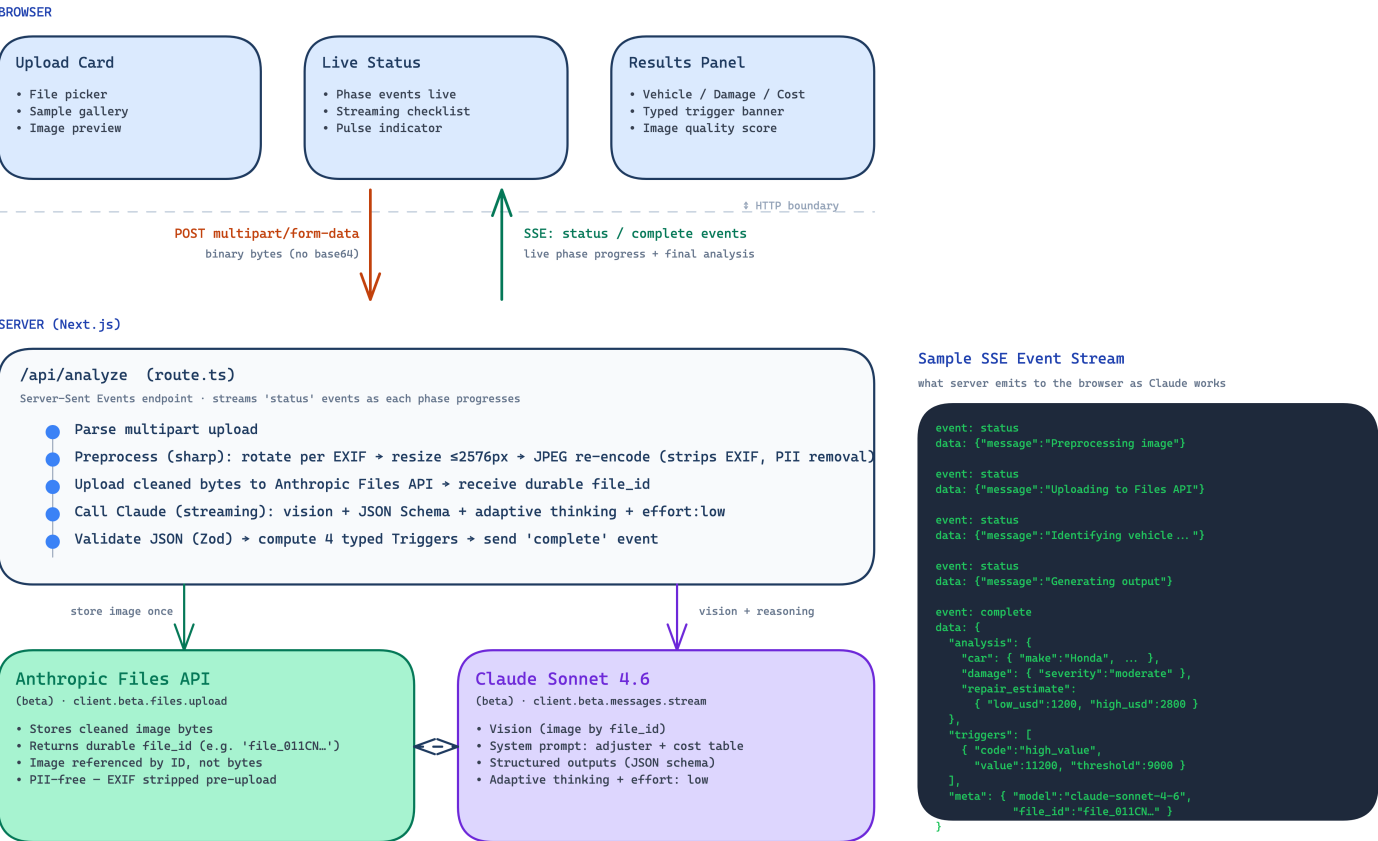


Figure 1. Logical layout. The browser holds the SPA, the server runs the SSE endpoint, and Anthropic's Files API + Claude API are the only external dependencies. Source: architecture.excalidraw (editable at excalidraw.com).

3. End-to-End Data Flow

Step-by-step trace of what happens when a user clicks Analyze. Total wall-clock time: approximately 5–10 seconds. Streaming makes the wait feel active rather than blocked.

#	Where	What happens
1	Browser	User selects file (or sample fetched as blob, wrapped as File)
2	Browser	POSTs multipart/form-data to /api/analyze
3	Server	Parses file from FormData
4	Server	Image preprocess (sharp): rotate per EXIF → resize to ≤ 2576 px → JPEG re-encode → strip EXIF metadata (PII removal)
5	Server	Opens SSE stream back to browser
6	Server	Sends status: "Preprocessing image"
7	Server	Uploads to Anthropic Files API → receives file_id
8	Server	Sends status: "Uploading to Anthropic Files API"
9	Server	Calls Claude Sonnet 4.6 (streaming): image by file_id, system prompt, JSON schema, adaptive thinking, effort: low
10	Server	Sends status: "Identifying vehicle and assessing damage"
11	Server	When Claude emits final text block: status "Generating structured output"
12	Server	Validates JSON output with Zod
13	Server	Computes typed Triggers (4 mechanical rules)
14	Server	Sends "complete" event with analysis + triggers + meta
15	Browser	Renders three result cards + structured trigger banner if any

4. AI Pipeline (the single Claude call)

The entire AI logic is one `client.beta.messages.stream()` call. No model chaining, no multi-step orchestration. Vision, reasoning, and structured output happen in a single request.

Field	Value	Purpose
model	claude-sonnet-4-6	Vision-capable, balanced for speed/quality
thinking	{ type: 'adaptive' }	Model decides reasoning depth dynamically
output_config.format	JSON Schema (ANALYSIS_JSON_SCHEMA)	Enforces output shape — no JSON parsing flakiness
output_config.effort	"low"	Speed-tuned reasoning for first-pass assessment
system	Adjuster persona + cost reference table	Steers behavior, anchors cost estimates
user content	Image (by file_id) + analysis instruction	Multimodal input
betas	["files-api-2025-04-14"]	Required for file_id image references

Output schema (every field guaranteed by the API)

Field path	Description
car.{make, model, color, year_estimate}	Vehicle identification
car.field_confidences.{make, model, color, year}	0–100 per-field confidence
car.confidence_reasoning	Why the model is more or less confident
damage.{summary, affected_areas[], severity}	Damage description and classification
damage.severity_confidence	0–100 confidence in severity classification
repair_estimate.{low_usd, high_usd, confidence, reasoning}	Cost range with reasoning
image_quality.{score, issues[]}	0–100 image quality assessment
requires_human_review, review_triggers[]	Model self-assessment (overridden server-side)
analysis_notes	Free-form summary for adjuster context

5. How Images Are Processed

The image traverses five stages from the user's hard drive to Claude's vision system. The middle stage (server-side preprocessing) is where the meaningful work happens — PII removal and dimension capping before any external service sees the bytes.

Stage	Where	What happens
1	Browser	User picks file via the file input or clicks a sample. Samples are fetched client-side and wrapped as a File object so they share the exact same upload path.
2	Browser → Server	FormData + native fetch sends the file as multipart/form-data — binary bytes over the wire, no base64 encoding.
3	Server	Request.formData() parses the multipart body, file.arrayBuffer() converts to a Node Buffer. Validation errors return as JSON 4xx, not in-stream errors.
4	Server (sharp pipeline)	rotate() — apply EXIF orientation. resize() — cap to ≤ 2576 px long-edge, preserve aspect, no upscaling. jpeg(quality:90, mozjpeg) — re-encode, dropping ALL EXIF metadata.
5	Server → Anthropic	Cleaned bytes uploaded once to Files API → file_id returned. Claude messages reference the image by file_id, not by re-sending bytes.

Order of operations matters

The sharp pipeline runs three transformations and the order is deliberate. .rotate() must come BEFORE the JPEG re-encode — otherwise the EXIF orientation tag is gone by the time we'd want to apply it, and Claude would see phone photos sideways. .resize() in the middle caps token cost (larger images cost more vision tokens). The .jpeg() re-encode is what actually strips EXIF — sharp's encoders don't carry metadata through.

Why EXIF stripping matters

EXIF metadata in phone photos can include GPS lat/long (where the photo was taken), timestamps (when), and device fingerprints (model and serial). For a claims pipeline, any of these can leak PII or contradict the claimed incident details. Stripping at the edge — before the cleaned bytes reach Anthropic's Files API — means Anthropic never stores the metadata in the first place. This is real compliance handling, not theatre.

What ends up where

Stage	Image state	Size (typical)	Stored where
1. Browser pick	Original phone JPEG with full EXIF	4–8 MB	Browser memory
2. POST	Original bytes over the wire	4–8 MB transferred	In flight
3. Buffer	Same bytes in Node	4–8 MB	Server memory
4. After sharp	Resized JPEG, no EXIF	~400–800 KB	Server memory
5a. Files API	Cleaned bytes uploaded	~400–800 KB	Anthropic Files storage
5b. Claude call	Reference by file_id only	~30 bytes (just the ID)	Request payload

6. Trust & Escalation Layer (server-side)

After Claude returns the analysis, the server runs deterministic rules and produces typed Trigger objects. Each trigger carries a code (for routing), the actual value, and the threshold that tripped the rule. Banners fire when `triggers.length > 0`.

Trigger code	Rule	Threshold
low_confidence	Any field confidence below threshold	< 30 / 100
severe_damage	Damage classified as severe	severity = severe
poor_image_quality	Image quality score below threshold	< 50 / 100
high_value	Repair estimate high end above threshold	> \$9,000

Why server-side instead of model-driven? Two reasons. First, deterministic enforcement — the four mechanical rules execute every time, regardless of how Claude phrases its self-assessment. Second, typed downstream consumption — fraud routing, queue assignment, audit logs, and analytics can act on the trigger code and value without parsing prose.

Thresholds are exported constants in `lib/schema.ts` (`REVIEW_THRESHOLDS`). Different carriers calibrate differently — a conservative carrier might raise the confidence threshold to 70; an aggressive automation strategy might raise the high-value threshold to \$20,000. This is the per-carrier tuning surface in production.

7. Tech Stack

Layer	Choice
Framework	Next.js 16 (App Router) + TypeScript
Styling	Tailwind CSS 4
AI	@anthropic-ai/sdk + Claude Sonnet 4.6 (vision + structured outputs + adaptive thinking + streaming)
Image storage	Anthropic Files API (beta)
Image processing	sharp
Schema validation	zod (runtime) + hand-written JSON Schema (Claude contract)
Streaming	Server-Sent Events (native fetch + custom SSE parser)
Hosting	Local-only (npm run dev)

8. Key Design Decisions

1. One AI service, one API call

Vision, reasoning, and structured output all happen in a single Claude request. No model chaining, no glue code, one provider to talk about with the customer.

2. Structured outputs over JSON parsing

The contract is enforced by the API. The frontend never crashes on malformed model output; every field is guaranteed.

3. Server-side triggers, client-side rendering

The four mechanical review rules execute deterministically server-side after Claude returns. Bulletproof, typed, auditable. The UI just renders them.

4. Streaming for UX, not throughput

Wall-clock time is approximately 5–10 seconds either way. Streaming makes the wait feel instant by showing live phase progress (preprocessing → upload → vehicle ID → output).

5. EXIF stripping before upload

Real PII concern — GPS coordinates, timestamps, device info. Done at the edge with sharp before the image hits Anthropic storage.

6. Anthropic Files API for image storage

Avoids introducing S3 / Vercel Blob and the associated CORS / presigned-URL plumbing for a 4-hour build. The `file_id` becomes the durable handle to the image.

7. No persistence in v1

The original spec does not require it. Adding a database costs hours and is invisible in the demo. Phase 1 in the SOW adds Postgres + audit log.

End of document. See README.md for setup instructions and SOW.md for the production roadmap.